

TP 3 : Services Angular et Récupération de Données via HTTP

Cours Angular - ISITCOM

Année Universitaire 2025-2026

Contexte et Objectifs

Dans les TP précédents, vous avez créé `ListeProduitsComponent` et `ProduitDetailsComponent` en utilisant des données statiques (tableaux codés en dur). **L'objectif de ce TP est de refactoriser (modifier) ces composants** pour qu'ils récupèrent les données depuis un serveur simulé.

- Simuler une API REST avec `json-server`.
- Créer un Service Angular pour centraliser les appels HTTP.
- Remplacer les données statiques par des *Observables*.

1 Préparation de l'environnement

1.1 Mise en place de json-server

1. Installez et lancez le serveur avec le fichier `db.json` contenant vos produits.

```
1 json-server --watch db.json --port 3000
2
```

1.2 Configuration HTTP

Dans `app.config.ts`, activez le client HTTP :

```
1 export const appConfig: ApplicationConfig = {
2   providers: [
3     provideRouter(routes),
4     provideHttpClient() // TODO : Ajouter cet appel
5   ]
6 };
```

2 Création du Service de Données

Générez le service `ProduitService`. Ce service sera responsable de toutes les interactions avec le backend.

```
1 ng generate service services/produit
```

Travail à faire : Complétez le service ci-dessous pour gérer le CRUD.

```

1 @Injectable({ providedIn: 'root' })
2 export class ProduitService {
3   // TODO 1 : Définir l'URL de l'API (http://localhost:3000/produits)
4   private apiUrl = '...';
5
6   // TODO 2 : Injecter HttpClient via le constructeur
7   constructor(/* ... */) { }
8
9   // TODO 3 : Méthode pour récupérer tous les produits (renvoie
10    Observable<Produit[]>)
11   getProduits(): Observable<Produit[]> {
12     // ... return this.http.get ...
13   }
14
15   // TODO 4 : Méthode pour récupérer un produit par son ID (renvoie
16    Observable<Produit>)
17   getProduitById(id: number): Observable<Produit> {
18     // ... Astuce : concaténer l'ID à l'URL
19   }
20
21   // TODO 5 : Méthode pour supprimer un produit
22   deleteProduit(id: number): Observable<void> {
23     // ...
24   }
25 }

```

3 Refactoring : Liste des Produits

Vous allez modifier `liste-produits.component.ts` pour qu'il n'utilise plus de tableau statique, mais qu'il appelle le service.

3.1 Classe TypeScript

```

1 export class ListeProduitsComponent implements OnInit {
2   produits: Produit[] = [];
3
4   // TODO 1 : Injecter ProduitService
5   constructor(/* ... */) {}
6
7   ngOnInit(): void {
8     // Appel initial
9     this.chargerProduits();
10  }
11
12  chargerProduits() {
13    // TODO 2 : S'abonner (subscribe) à la méthode getProduits() du
14    service
15    // et mettre à jour this.produits avec le résultat
16  }
17
18  supprimer(id: number) {
19    // TODO 3 : Appeler la méthode delete du service
20    // Une fois la suppression réussie (dans le subscribe),
21    // mettre à jour la liste locale pour faire disparaître l'élément.
22  }
23 }

```

```
22 }
```

3.2 Template HTML

Assurez-vous que votre template utilise correctement les directives structurales.

```
1 <!-- TODO : Adaptez votre ancien template pour itérer sur le tableau '
   produits' -->
2 <!-- qui est maintenant rempli de manière asynchrone -->
3
4 <div *ngFor="let p of produits">
5   <!-- Affichage des infos du produit -->
6
7   <!-- Bouton vers les détails -->
8   <button [routerLink]="['/produits', p.id]">Détails</button>
9
10  <!-- Bouton de suppression avec liaison d'événement -->
11  <button (click)="supprimer(p.id)">Supprimer</button>
12 </div>
```

4 Refactoring : Détails du Produit

Modifiez `produit-details.component.ts` pour récupérer l'ID depuis l'URL, puis demander les infos au serveur.

4.1 Classe TypeScript

```
1 export class ProduitDetailsComponent implements OnInit {
2   produit: Produit | undefined;
3
4   // TODO 1 : Injecter ActivatedRoute (pour l'URL) et ProduitService
5   constructor(
6     private route: ActivatedRoute,
7     private produitService: ProduitService
8   ) {}
9
10  ngOnInit(): void {
11    // TODO 2 : Récupérer l'id depuis l'URL
12    const id = Number(this.route.snapshot.paramMap.get('id'));
13
14    if (id) {
15      // TODO 3 : Appeler getProduitById(id) du service
16      // et affecter le résultat à this.produit
17    }
18  }
19 }
```

4.2 Template HTML

```
1 <!-- TODO : Utiliser *ngIf pour n'afficher la section que si 'produit'
   est chargé -->
2 <div *ngIf="produit" class="detail-card">
3
```

```

4      <h2>{{ produit.nom }}</h2>
5
6      <!-- Affichez ici l'image, le prix, la description, etc. -->
7
8      <button routerLink="/produits">Retour à la liste</button>
9  </div>
10
11 <div *ngIf="!produit">
12   <p>Chargement en cours ou produit introuvable...</p>
13 </div>

```

5 Validation

1. Vérifiez que la liste se charge au démarrage de l'application.
2. Cliquez sur "Détails" d'un produit : vous devez voir ses informations spécifiques.
3. Supprimez un produit : il doit disparaître de la liste (et du fichier `db.json`).